

Base de données orienté graphe – Neo4J

IUT Lumière Lyon 2 – Laboratoire ERIC

Juba AGOUN

Introduction

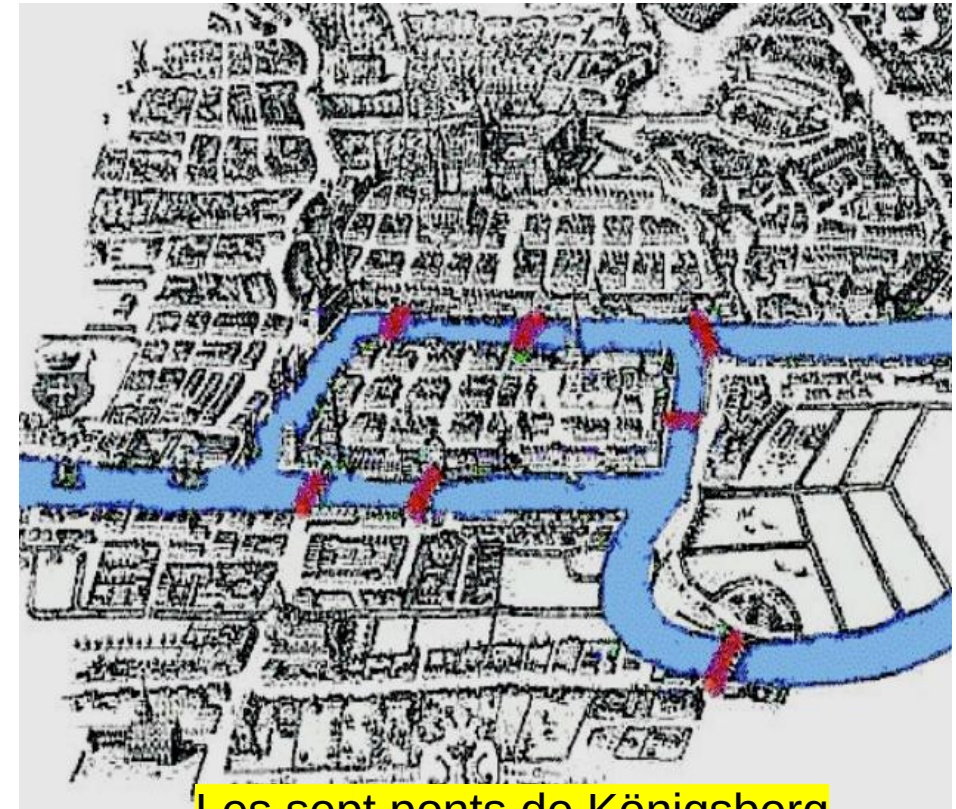
- Concept de connexion omniprésent
 - Réseaux sociaux, internet, biologie, transports
- Modèle d'analyse des bases de données NoSQL basé sur les agrégats
- Stocker des entités connectées est un challenge non adressé ni par les SGBD relationnels, ni par les bases NoSQL
- Relations difficiles à modéliser par :
 - Les BD relationnelles
 - Les autres modèles de BD NoSQL

Base de données orienté graphe – Neo4J

Les graphes permettent une modélisation naturelle des relations entre entités

Les bases NoSQL orientées graphes

- Les bases de données orientées graphes sont basées sur la théorie mathématique des graphes qui a été inventée par le mathématicien Leonhard Euler
- Permettent la modélisation, le stockage et la manipulation de données complexes et fortement connectées
- Utilisent un moteur de stockage pour les objets (similaire à une base documentaire, chaque entité de cette base étant nommée nœud), et un mécanisme de description d'arcs (relations entre les objets), arcs orientés et avec propriétés (nom, date, ...).



Les sept ponts de Königsberg

Les habitants de Königsberg se demandaient s'il était possible, en partant d'un quartier quelconque de la ville, de traverser tous les ponts sans passer deux fois par le même et de revenir à leur point de départ

Qu'est ce que c'est qu'un graphe?

- Les graphes contiennent des nœuds, des arêtes et des propriétés, qui sont tous utilisés pour représenter et stocker les données d'une manière que les bases de données relationnelles ne sont pas équipées pour le faire.
- Il existe beaucoup de modèles pour représenter les relations entre les données dans les SGBD graphe. Cependant, un certain effort a été fait pour créer le Modèle de Graphe Attribué (Property Graph Model), unifiant la plupart des différentes implémentations de graphes.
- Performances en lecture et parcours des données connectées

Rappel mathématique sur les graphes et leur propriété

Qu'est ce que c'est qu'un graphe?

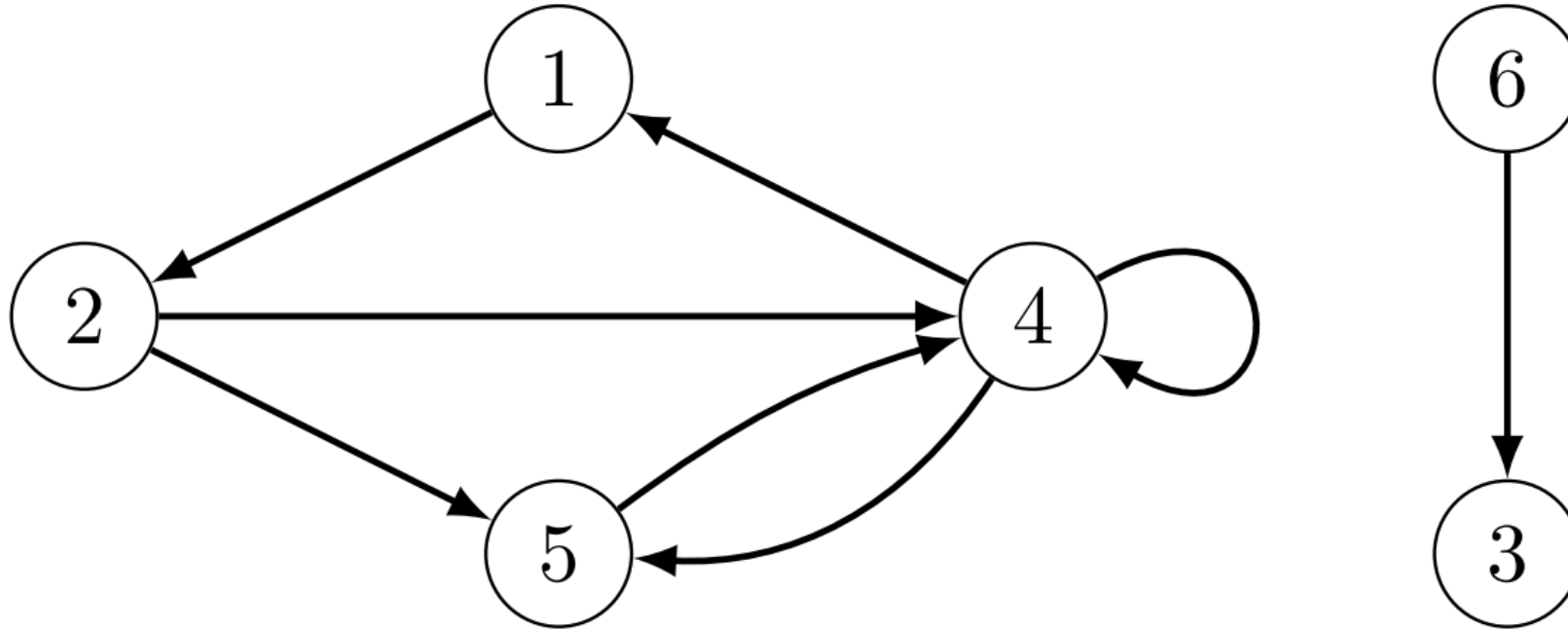
Définition

Un **graphe** orienté (resp. non orienté) G est la donnée d'un couple $G = (S, A)$ tel que :

- S est un ensemble fini de sommets,
- A est un ensemble de couples ordonnés (resp. non ordonnés) de sommets $\{s_i, s_j\} \in S^2$.

- ❖ Un couple (s_i, s_j) est appelé un **arc**, et est représenté graphiquement par $s_i \rightarrow s_j$, s_i est le sommet initial ou origine, et s_j le sommet terminal ou extrémité.
- ❖ L'arc $a = (s_i, s_j)$ est dit **sortant** en s_i et **incident** en s_j , et s_j est un **successeur** de s_i , tandis que s_i est un **prédécesseur** de s_j .
- ❖ L'ensemble des successeurs d'un sommet $s_i \in S$ est noté $Succ(s_i) = \{s_j \in S, (s_i, s_j) \in A\}$.
L'ensemble des prédécesseurs d'un sommet $s_i \in S$ est noté $Pred(s_i) = \{s_j \in S, (s_j, s_i) \in A\}$.

Graphe orienté – Exemple



Le graphe orienté $G = (S, A)$ avec :

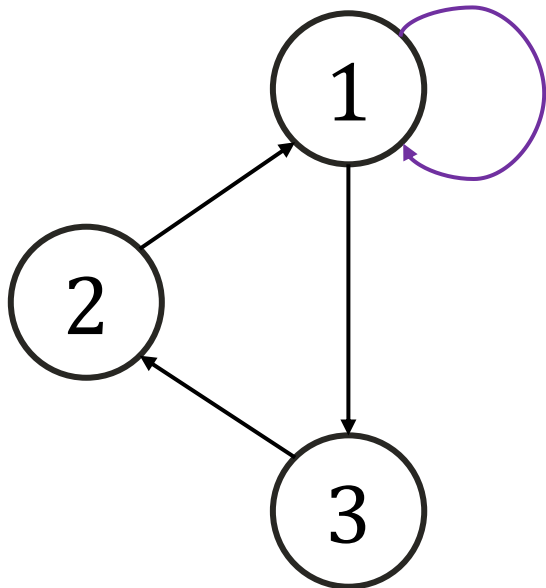
- $S = \{1, 2, 3, 4, 5, 6\}$ et,
- $A = \{(1, 2), (2, 4), (2, 5), (4, 1), (4, 4), (4, 5), (5, 4), (6, 3)\}$.

Théorie des graphes - Graphe Orienté

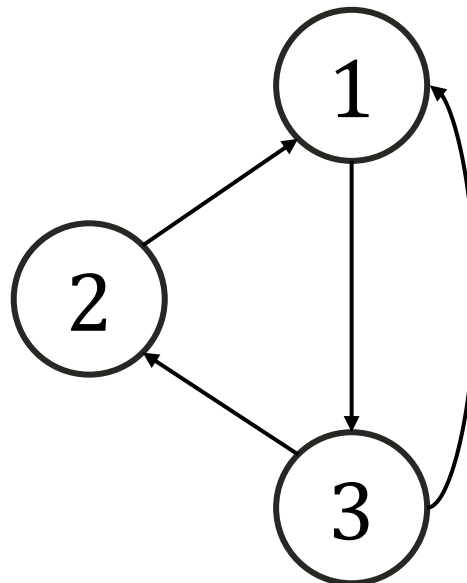
Définition

Une **boucle** est un arc reliant un sommet à lui-même. Un graphe orienté est dit **élémentaire** s'il ne contient pas de boucle.

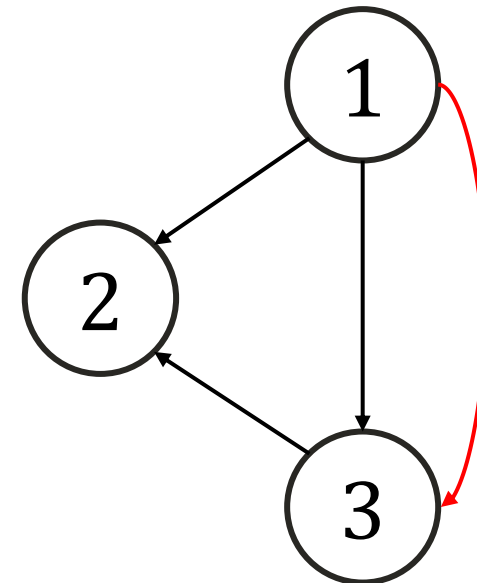
Un graphe orienté est un **p-graphe** s'il comporte au plus p arcs entre deux sommets. Le plus souvent, on étudiera des 1-graphes.



Boucle

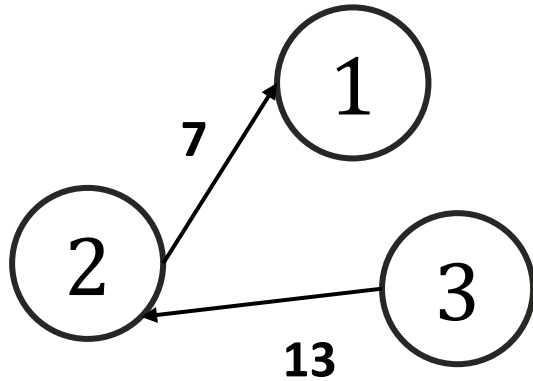


1-graphe



1-graphe

Théorie des graphes - Différents types de graphes

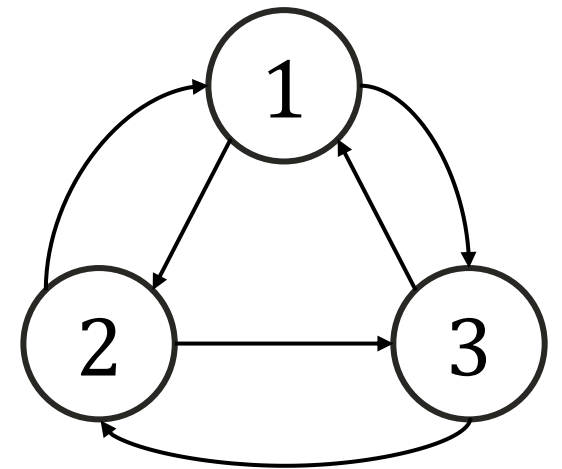


Définition (graphe valué)

Un graphe valué $G = (S, A, \nu)$ est un graphe (S, A) (orienté ou non-orienté) muni d'une application $\nu : A \rightarrow R$. L'application ν est appelée valuation du graphe. On peut étendre cette valuation en posant $\forall (x, y) \in S^2, \nu(x, y) = +\infty$ si $(x, y) \notin A$.

Définition (graphe complet)

Un 1-graphe orienté élémentaire est dit complet s'il comporte un arc (s_i, s_j) et un arc (s_j, s_i) pour tout couple de sommets différents $s_i, s_j \in S$.



Les bases NoSQL orientées graphes

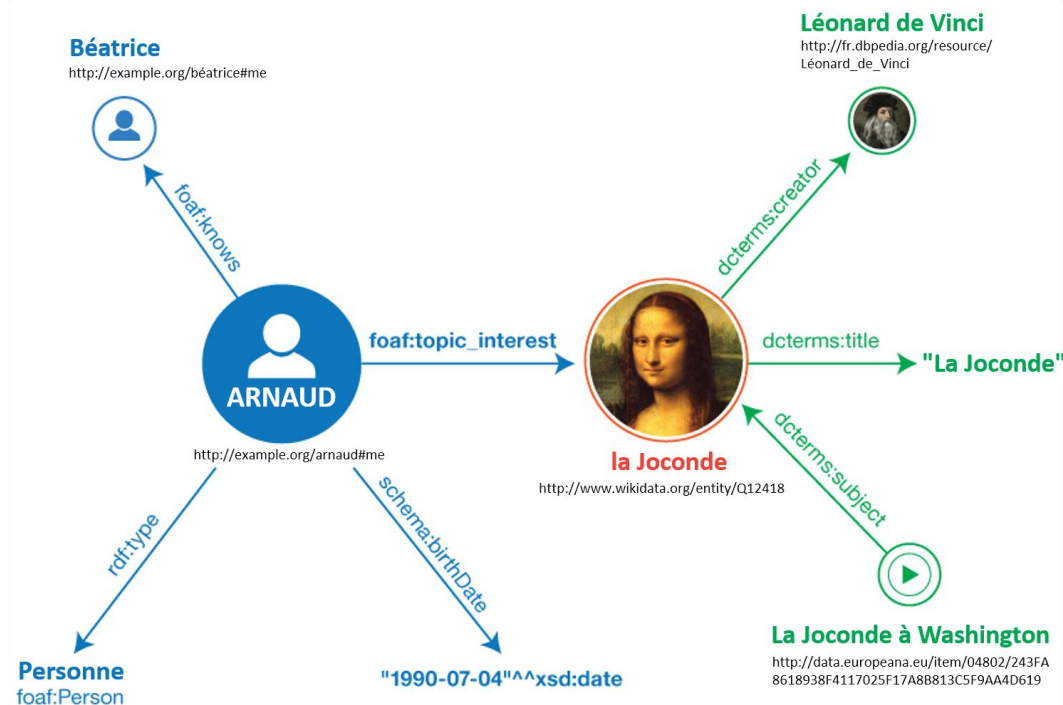
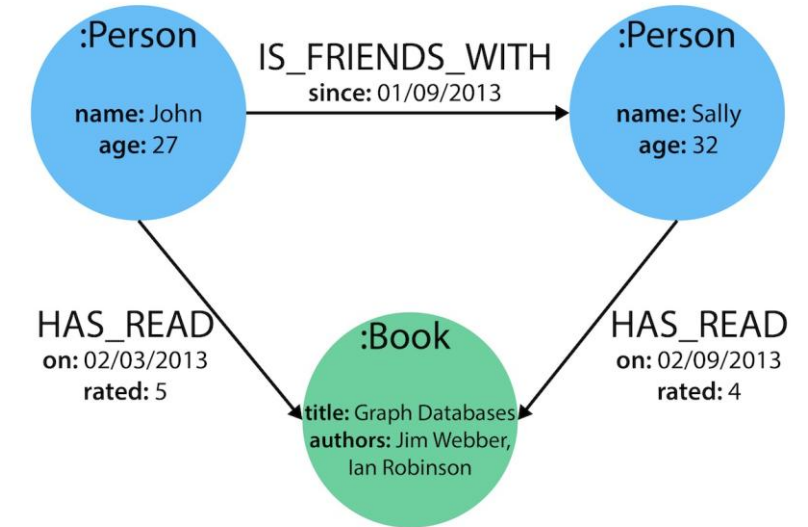
Les bases NoSQL orientées graphes

- Ces bases de données sont optimisées pour des recherches exploitant la localité des données, à partir d'un ou plusieurs nœuds racines, plutôt que des recherches globales.
 - Problèmes avec des jointures
 - Evolution constante du jeu de données
- Intérêts :
 - L'abstraction en graphes permet de résoudre des problématiques courantes, (cartographie, relations entre personnes, ...), avec des performances en lecture et parcours des données connectées
 - Modèle de données puissant plus proche du naturel et permet d'ajouter une sémantique aux relations, typer les nœuds, définir des attributs sur les nœuds et les liens

Types de base de données orientée graphe

➤ Graphes de propriétés

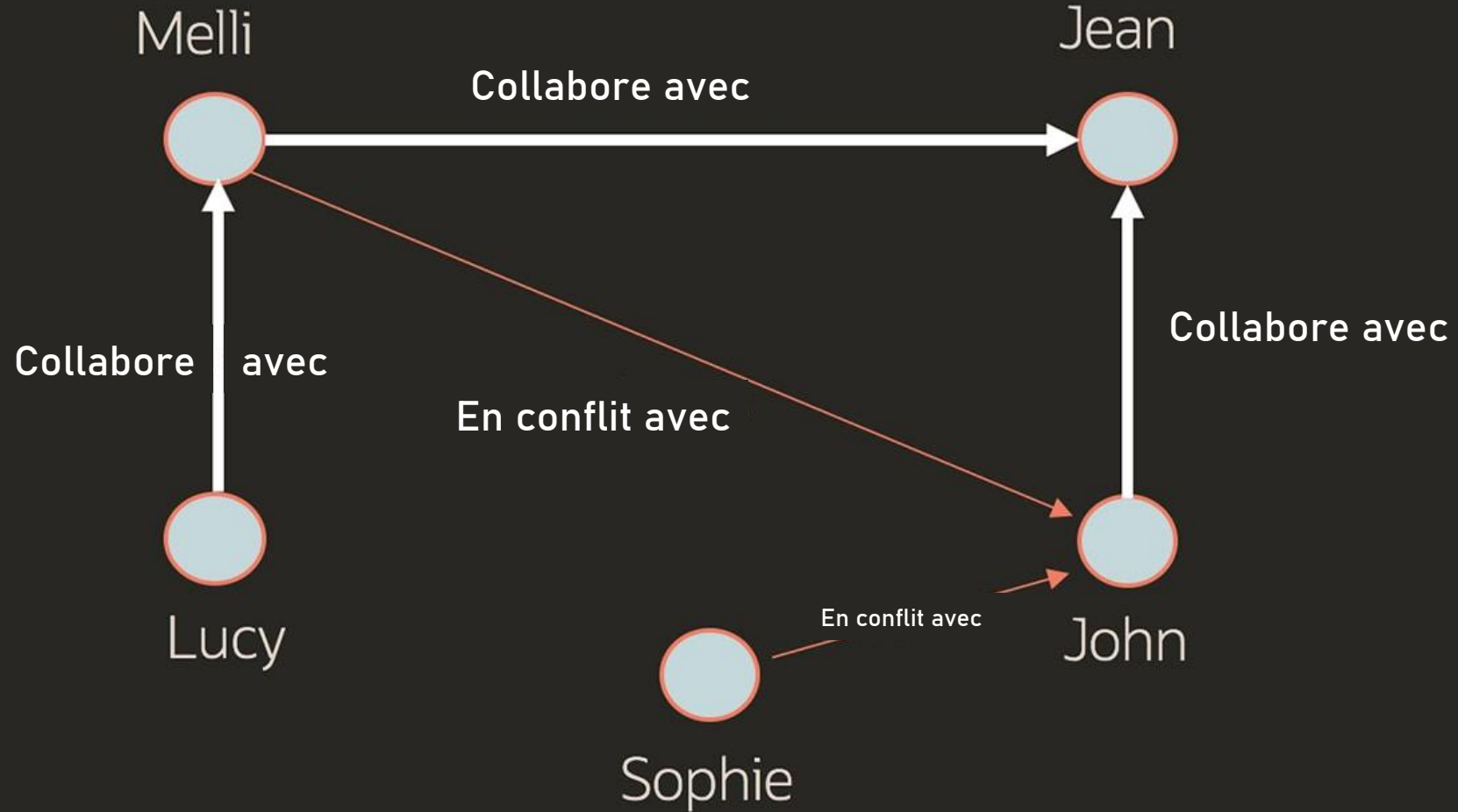
➤ Graphes RDF



Graphe de propriétés

- Les graphes de propriétés sont utilisés pour **modéliser les relations entre les données**, et ils permettent **d'effectuer des requêtes et des analyses de données basées sur ces relations**.
- Un graphe de propriétés comporte des **sommets** qui **peuvent contenir des informations détaillées sur un sujet**, et des **arêtes** qui indiquent la **relation entre les sommets**.
- Les sommets et les arêtes peuvent **avoir des attributs, appelés propriétés**, auxquels ils sont associés.

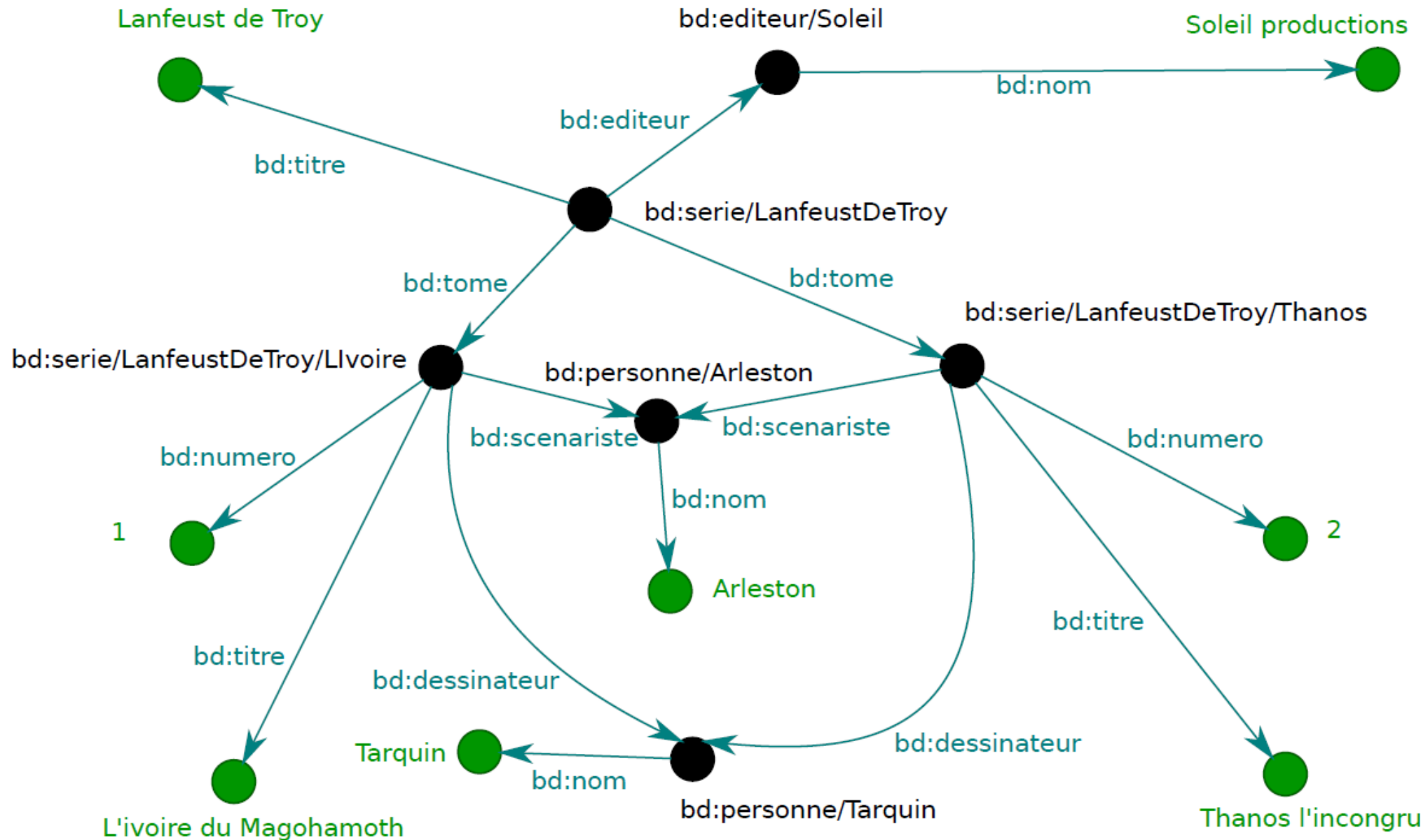
Graphe de propriétés - Exemple



Graphes RDF

- Les graphes RDF (*Resource Description Framework*) sont conformes à un ensemble de normes du W3C (*Worldwide Web Consortium*) conçues pour représenter des **énoncés et sont les mieux adaptés pour représenter des métadonnées et des données** de base complexes.
- Ils sont souvent utilisés pour les **données liées (linked data)**, l'intégration des données et les **graphes de connaissances**. Ils peuvent représenter des **concepts complexes** dans un domaine, ou **fournir une sémantique riche** et une **inférence sur les données**.

Graphes RDF



Les bases de données orientées graphes - Usages

- Manipulation d'objets complexes organisés en réseaux : cartographie, réseaux sociaux, ..
- Moteurs de recommandations
- Semantic Web
- Social computing
- Données géospatiales
- Généalogie
- Web of things
- Catalogue des produits
- Sciences de la Vie et calcul scientifique (bioinformatique, ...)
- Données liées, données hiérarchiques
- Services de routage, d'expédition et de géolocalisation

Les bases de données orientées graphes - Limites

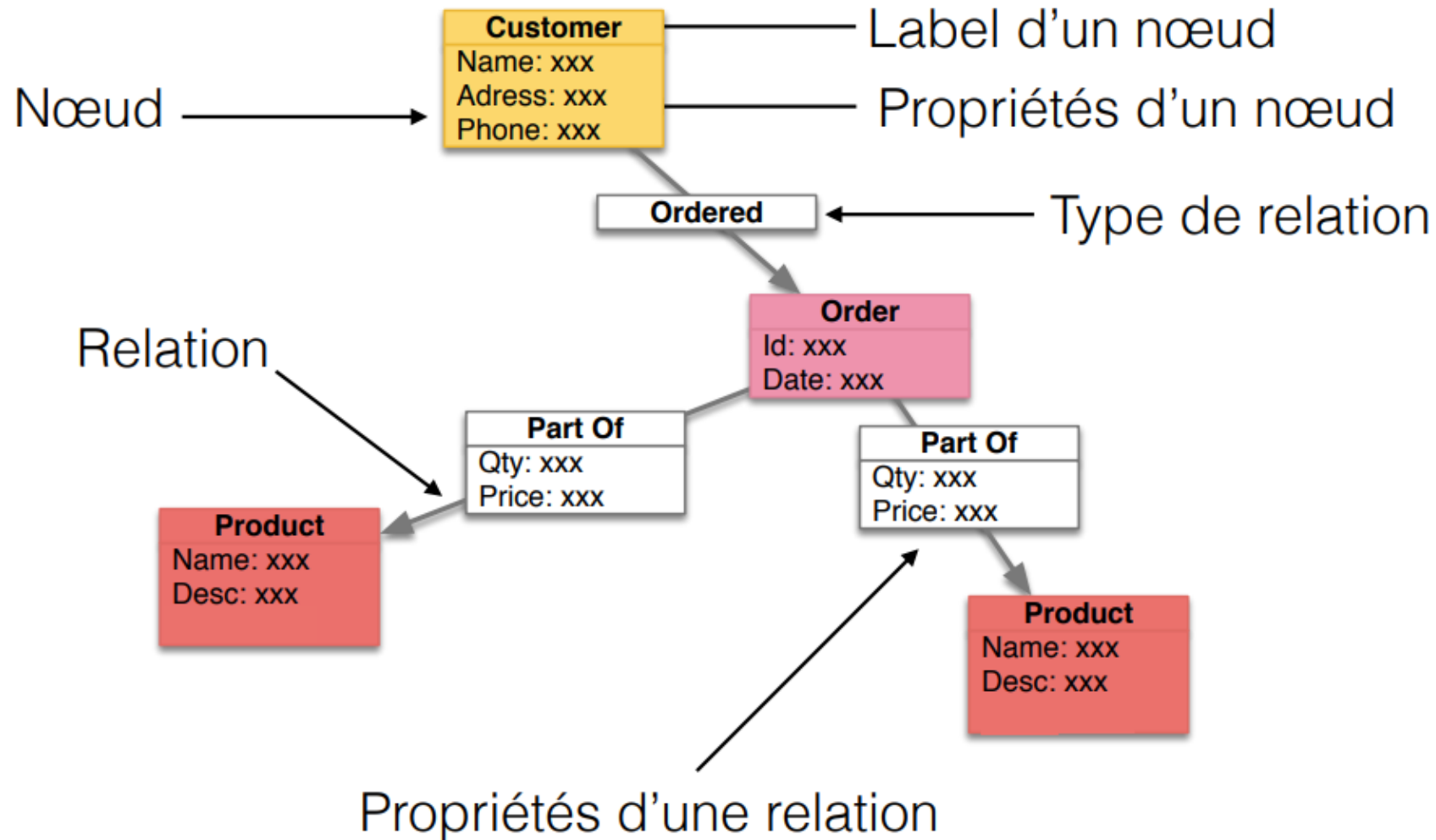
- Scalabilité moindre : Les mécanismes de synchronisation des écritures sur des graphes nécessite des opérations complexes.
- Ces systèmes sont donc fragiles au partitionnement des données sur des supports physiques distincts.

Modélisation de données en graphe

Modélisation de données en graphe

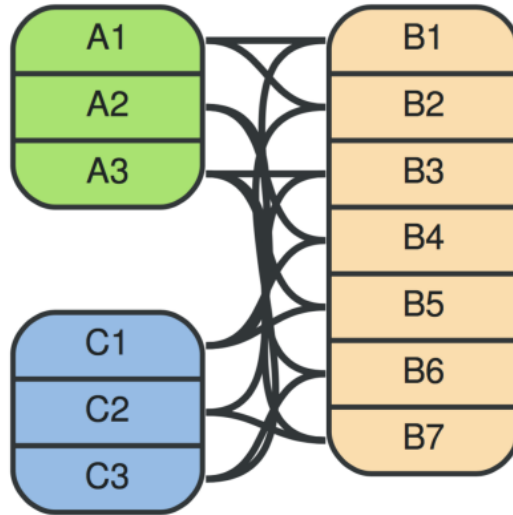
- La modélisation des données est la traduction d'une vue conceptuelle de vos données vers un modèle logique.
- Les BD orientées graphe se basent généralement sur les concepts de :
 - **Noeud** : une entité
 - **Label** : permet de regrouper des nœuds entre eux
 - **Relation** : matérialise un lien (dirigé) entre deux nœuds, possède un type
 - **Propriétés** : un nœud ou une relation peuvent disposer de propriétés (numériques, chaînes de caractères, booléens ou une liste des précédents types)

Modélisation de données en graphe

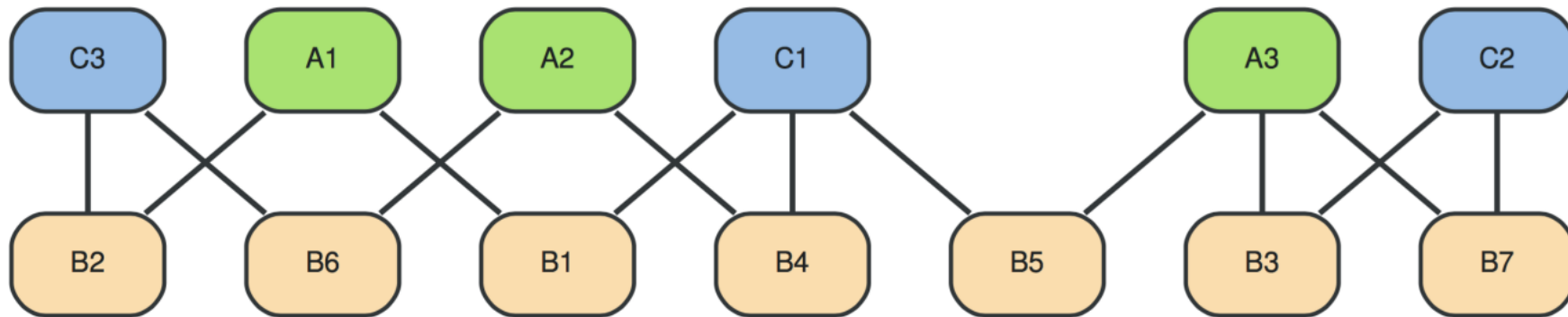


Modélisation de données en graphe - Du relationnel au graphe

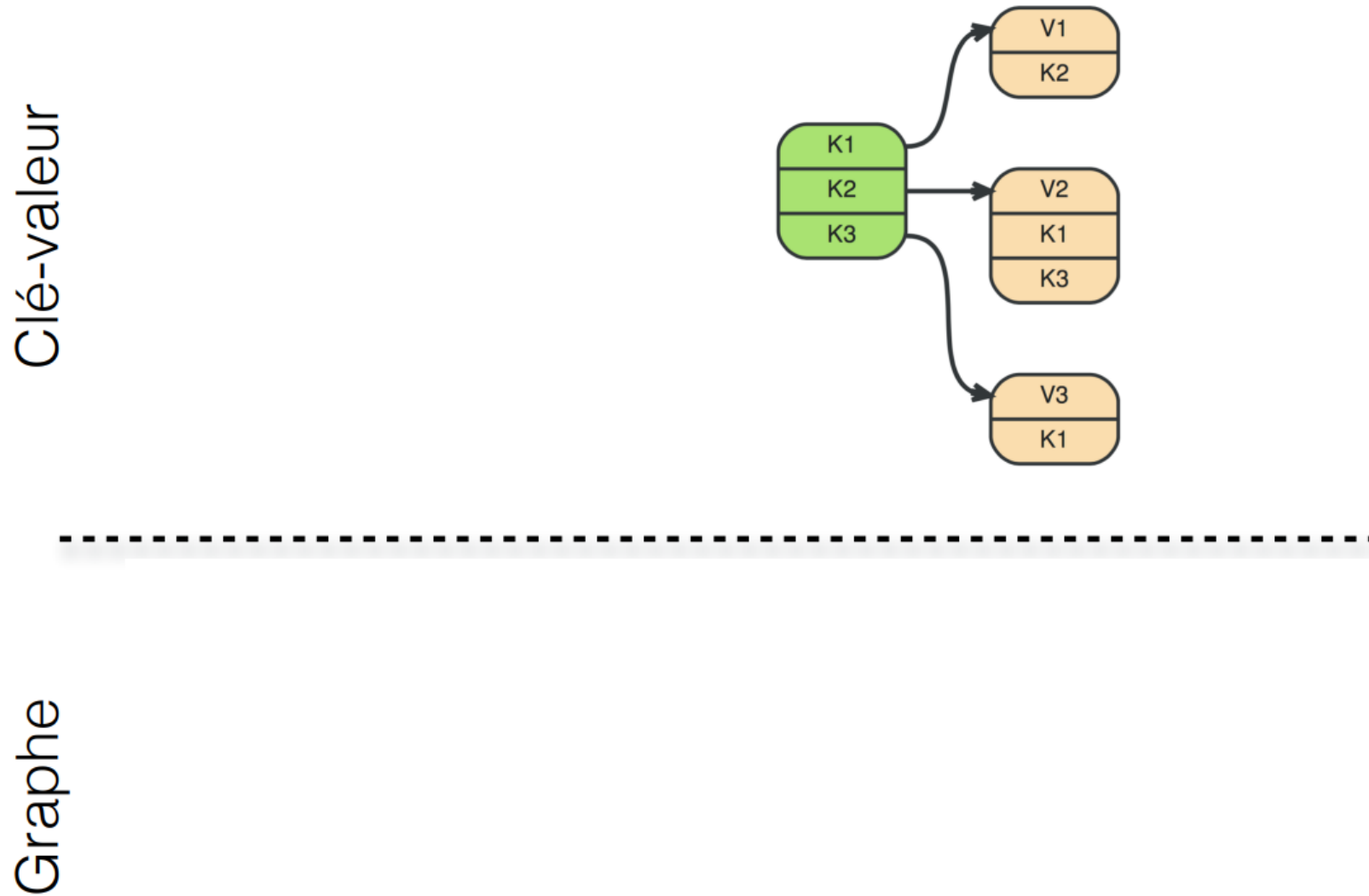
Relationnel



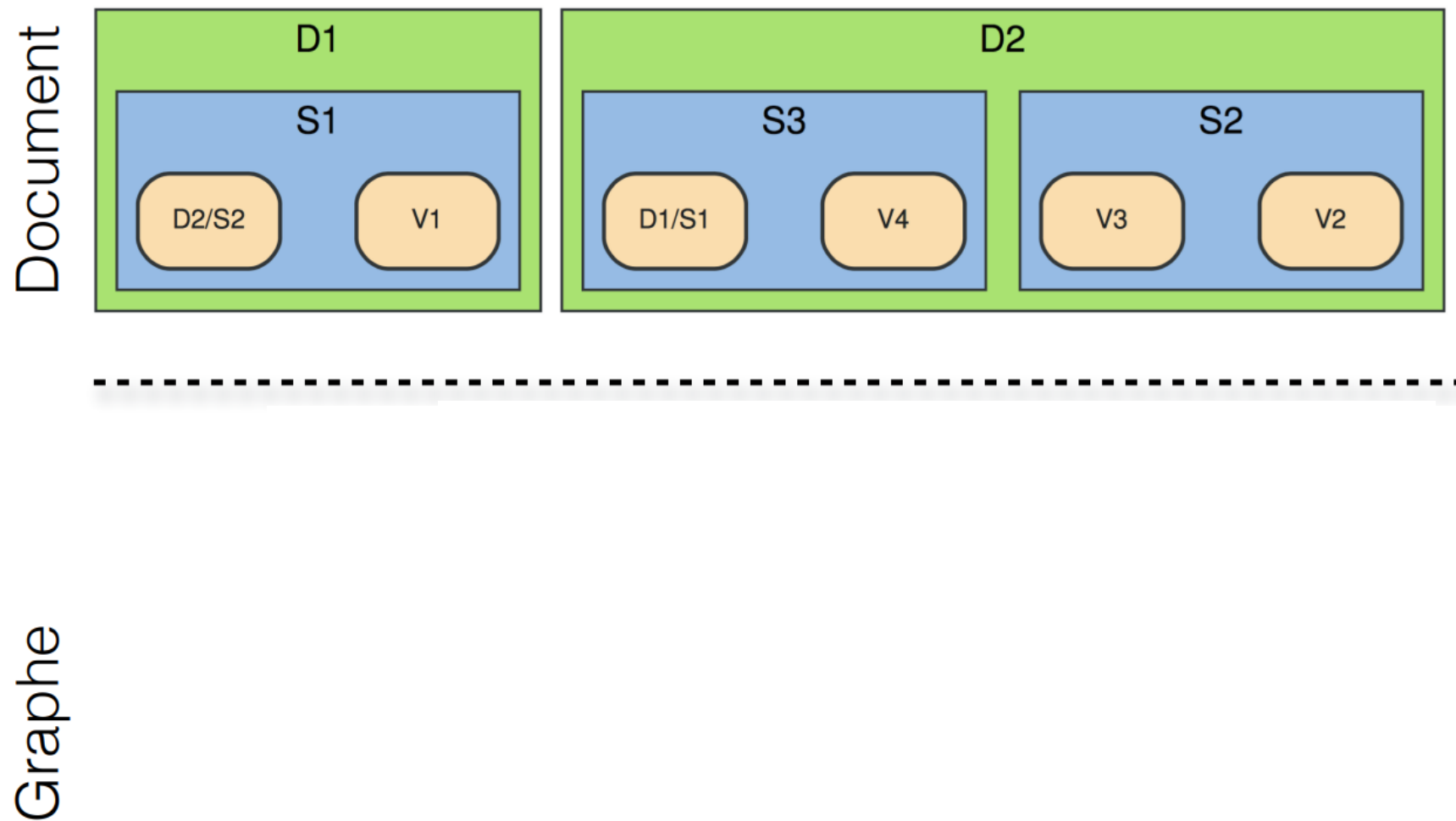
Graphe



Modélisation de données en graphe - Du relationnel au graphe



Modélisation de données en graphe - Du relationnel au graphe



Modélisation de données en graphe - Du relationnel au graphe

Relationnel	Customer		Product		Orders		
	Id	Name	Id	Name	Id	CustomerId	ShippingAdressId
	1	Martin	27	NoSQL Distilled	99	1	77
	Address		OrderItem				
	Id	City	Id	OrderId	ProductId	Price	
	77	Chicago	100	99	27	32.45	

Graphe

Modélisation de données en graphe

Représentation d'une relation:

➤ Ann aime Dan

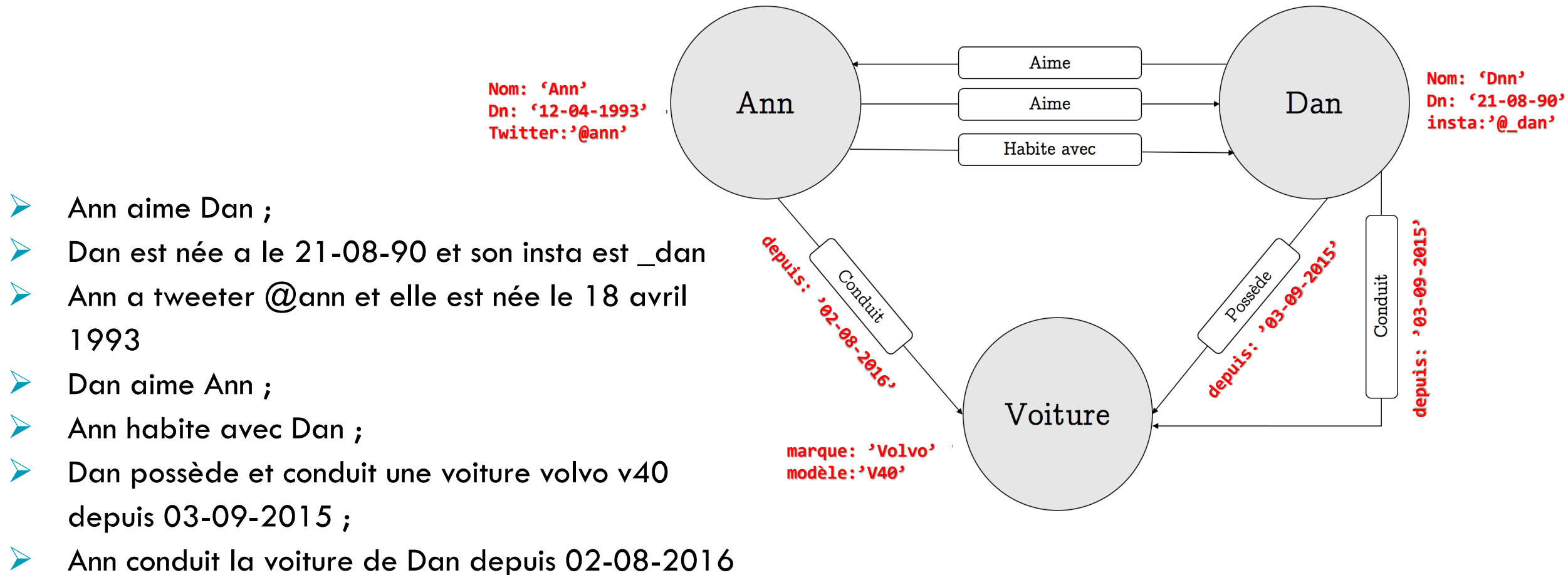
Modélisation de données en graphe

Les relations sont orientées

- Ann aime Dan ;
- Dan aime Ann ;
- Ann habite avec Dan;

Modélisation de données en graphe

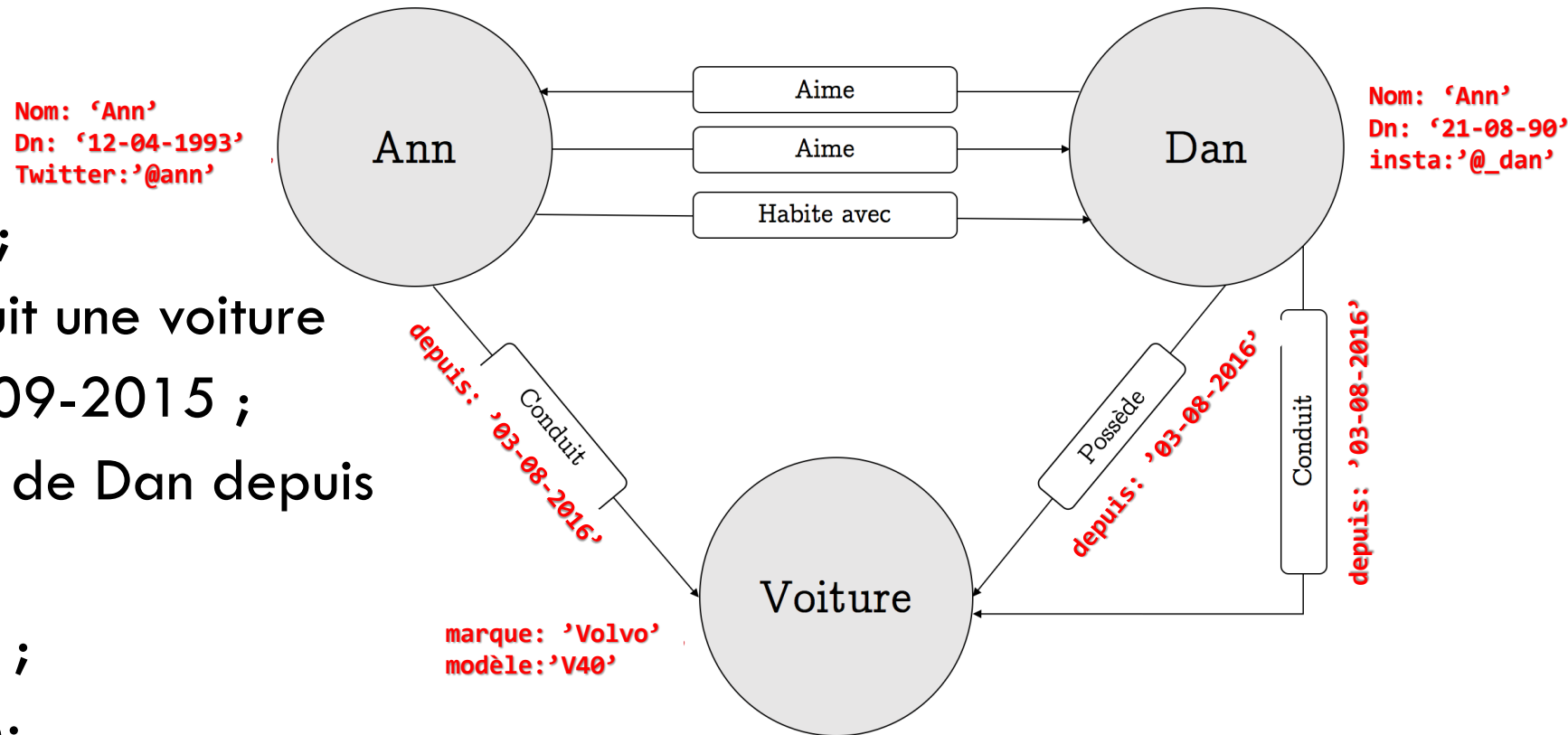
Les propriétés des nœuds et des relations sont des ensembles de clé:valeur



Modélisation de données en graphe

La représentation des classes se fait à travers des étiquettes de type clé:valeur

- Ann aime Dan ;
- Dan aime Ann ;
- Ann habite avec Dan ;
- Dan possède et conduit une voiture volvo v40 depuis 03-09-2015 ;
- Ann conduit la voiture de Dan depuis 02-08-2016
- Ann est une :personne ;
- Dan est une :personne;
- la volvo v40 est une :voiture



Modélisation de données en graphe - Résumé

➤ Nœud

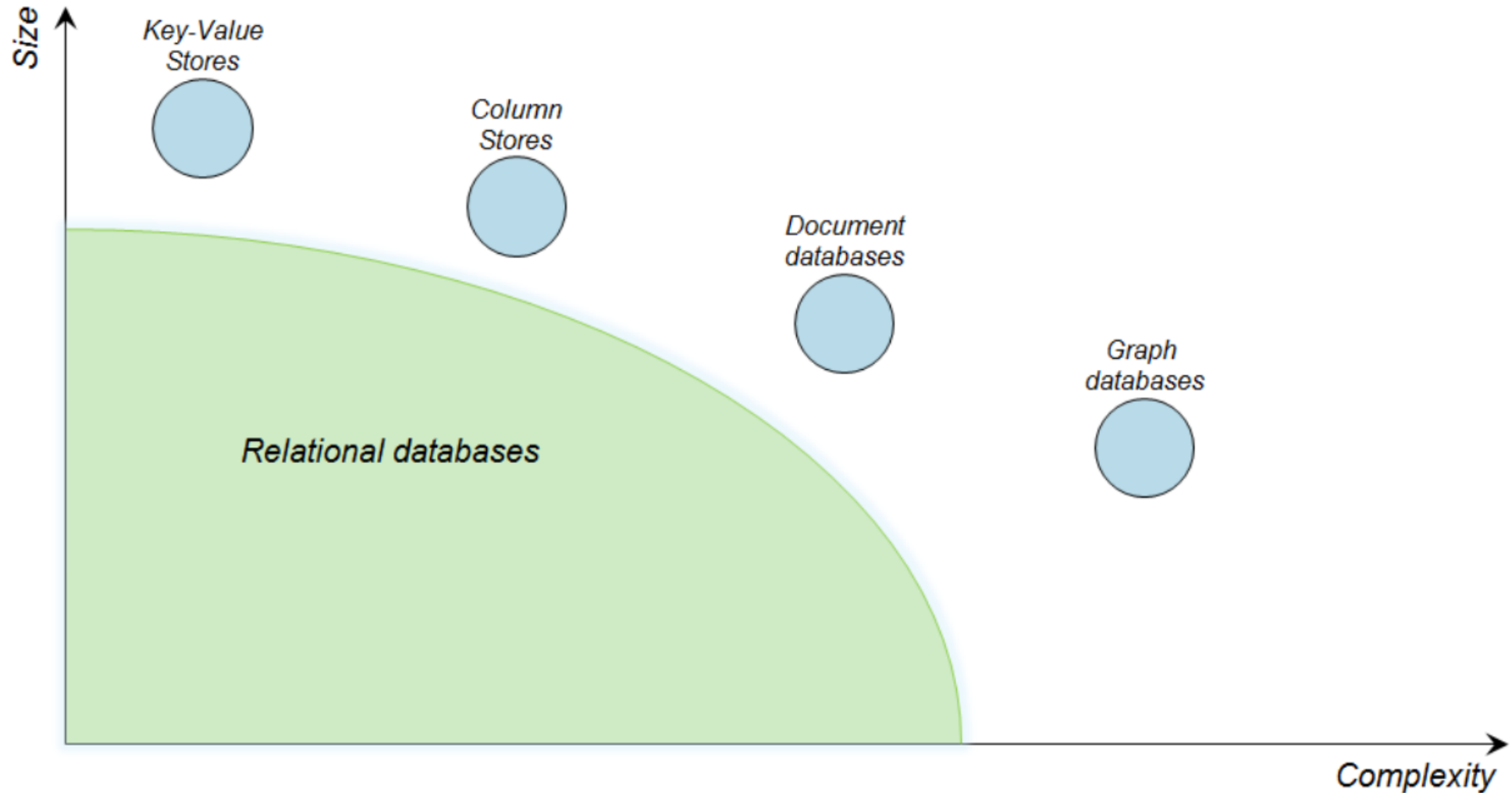
- un objet du graphe
- peut avoir des propriétés
- peut être étiqueté

➤ Relation

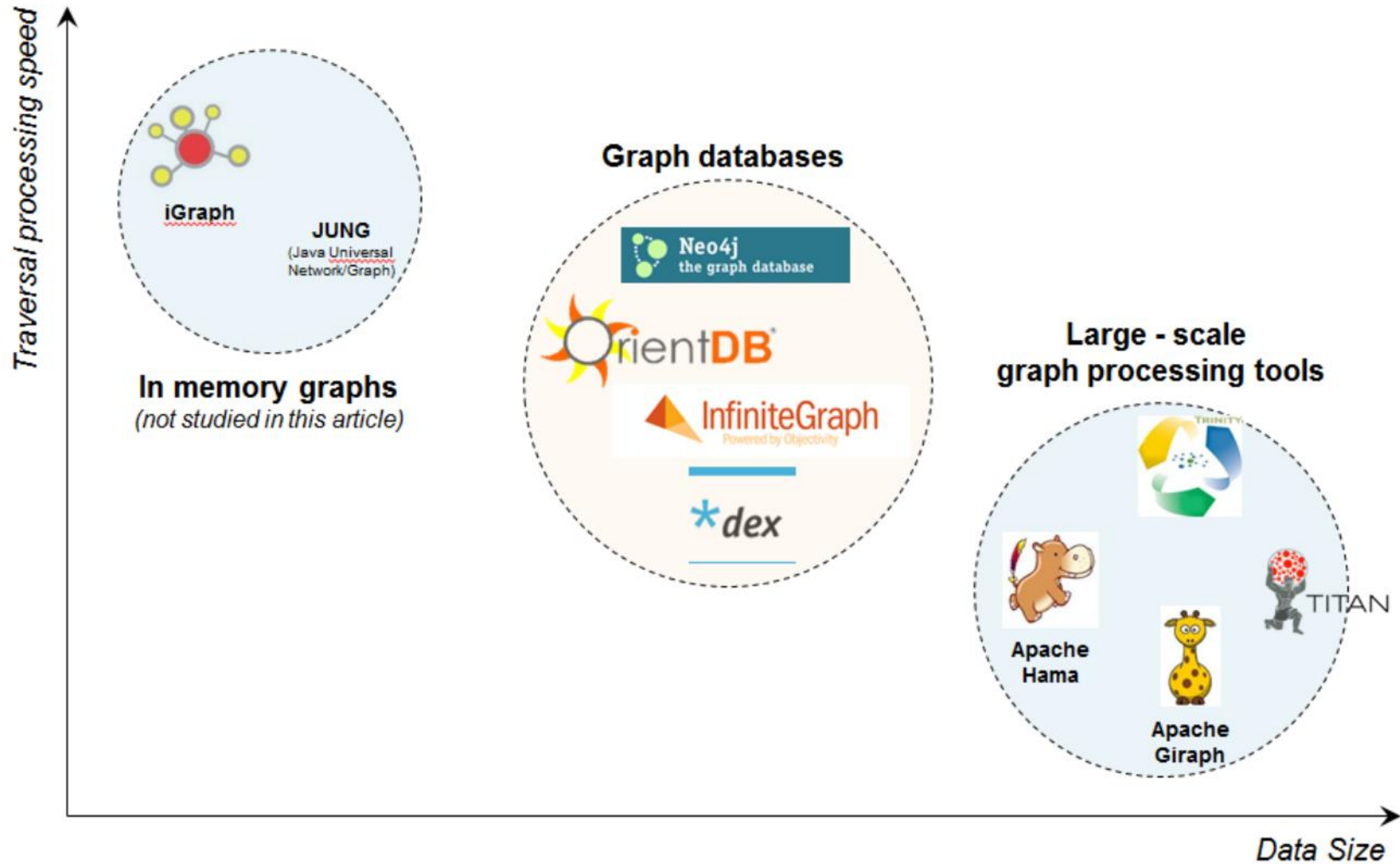
- un objet du graphe
- relie les nœuds par un type et une direction
- peut avoir des propriétés

Position dans l'écosystème NoSQL

Position dans l'écosystème NoSQL

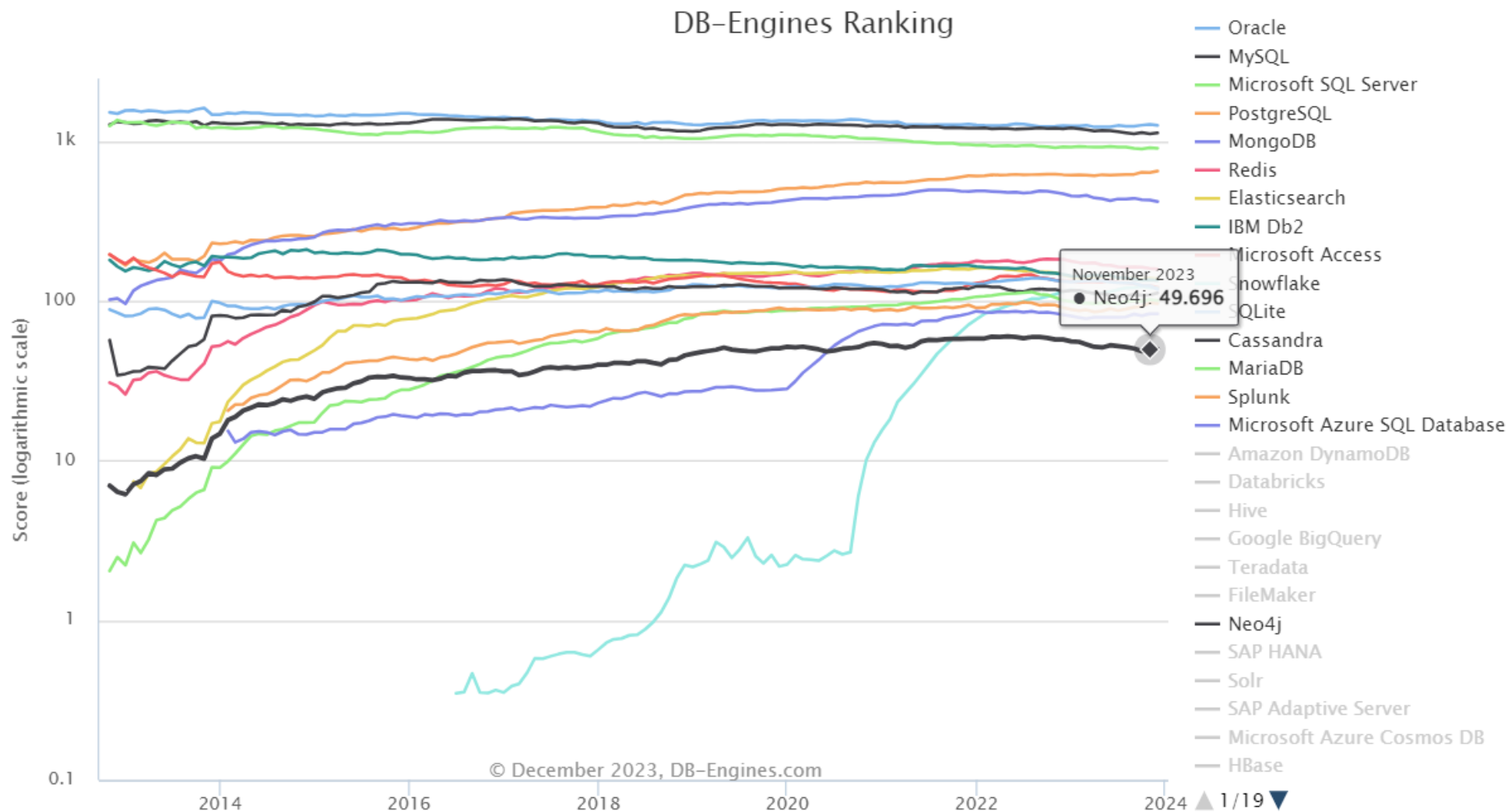


Etat des lieux



Etat des lieux

22^{ème} position selon <https://db-engines.com/>



Etat des lieux

➤ **Neo4J**

- Licence GPL, ACID compliant, basé sur Java

➤ **Infinite Graph**

- Propriétaire (Objectivity), passage à l'échelle virtuellement illimité, utilisé à la CIA et au département de la défense américain

➤ **AllegroGraph**

- Propriétaire (Franz Inc.), linked data et Web sémantique, supporte SPARQL, RDFS++ et Prolog

➤ **FlockDB**

- Créé par Twitter, pas de version stable, requêtes limitées

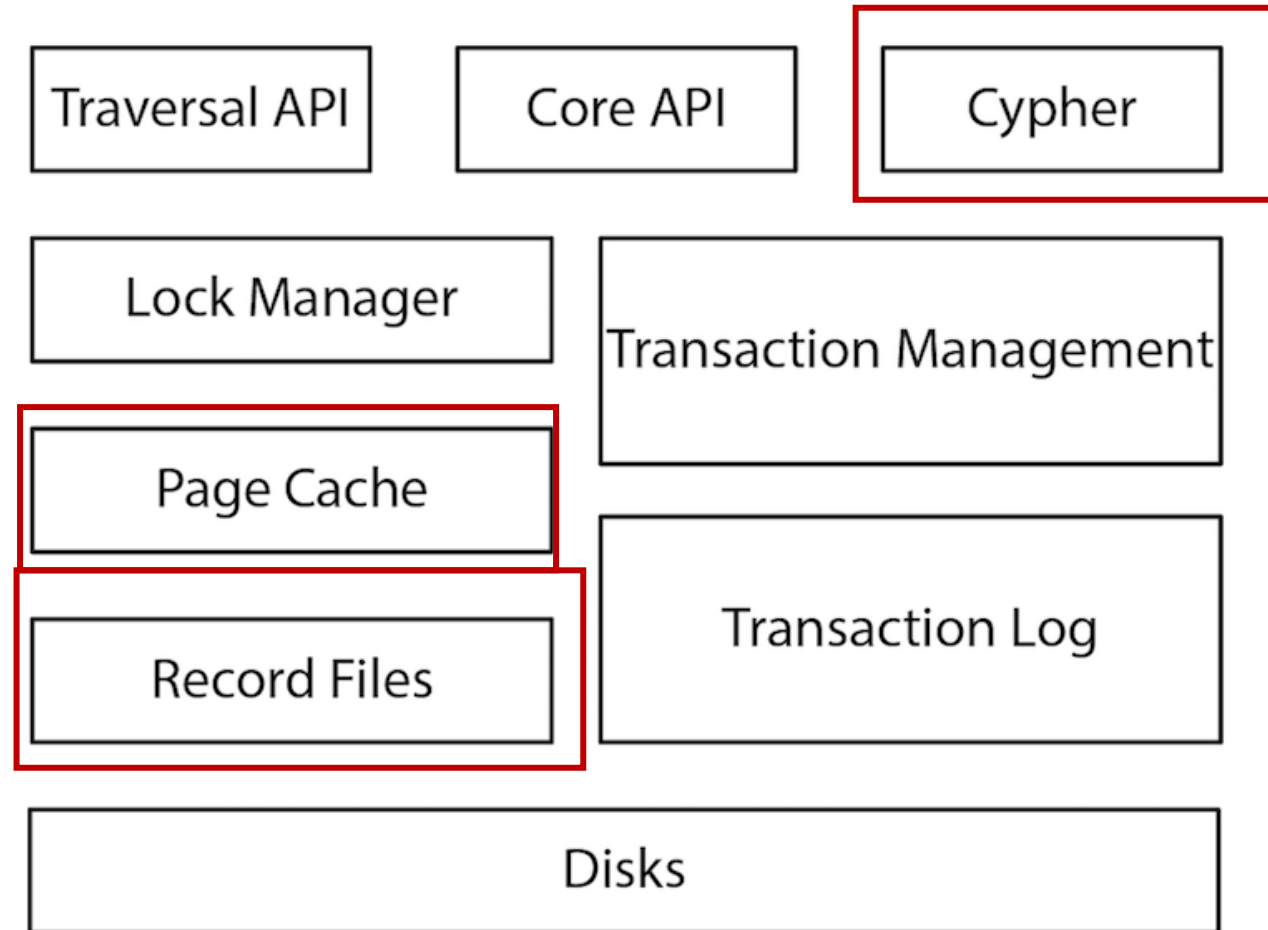
Neo4j

Quelques faits sur Neo4J

- Logiciel libre (licence GPLv3), Java, projet initié en 2000 (version 1.0 en 2010)
- Une des BD graphes les plus évoluées et les plus robustes, version courante : 4.4
- **Transaction**
 - Une base de données transactionnelle, respectueuse des principes ACID
- **Haute disponibilité**
 - Grace à la mise en place d'un cluster
- **Volumétrie**
 - Permet de stocker et requêter des milliards de nœuds et de relations
- **Cypher**
 - Un langage de requête graphe déclaratif, simple et efficace
- **Schemaless**
 - Pas de schéma préétabli

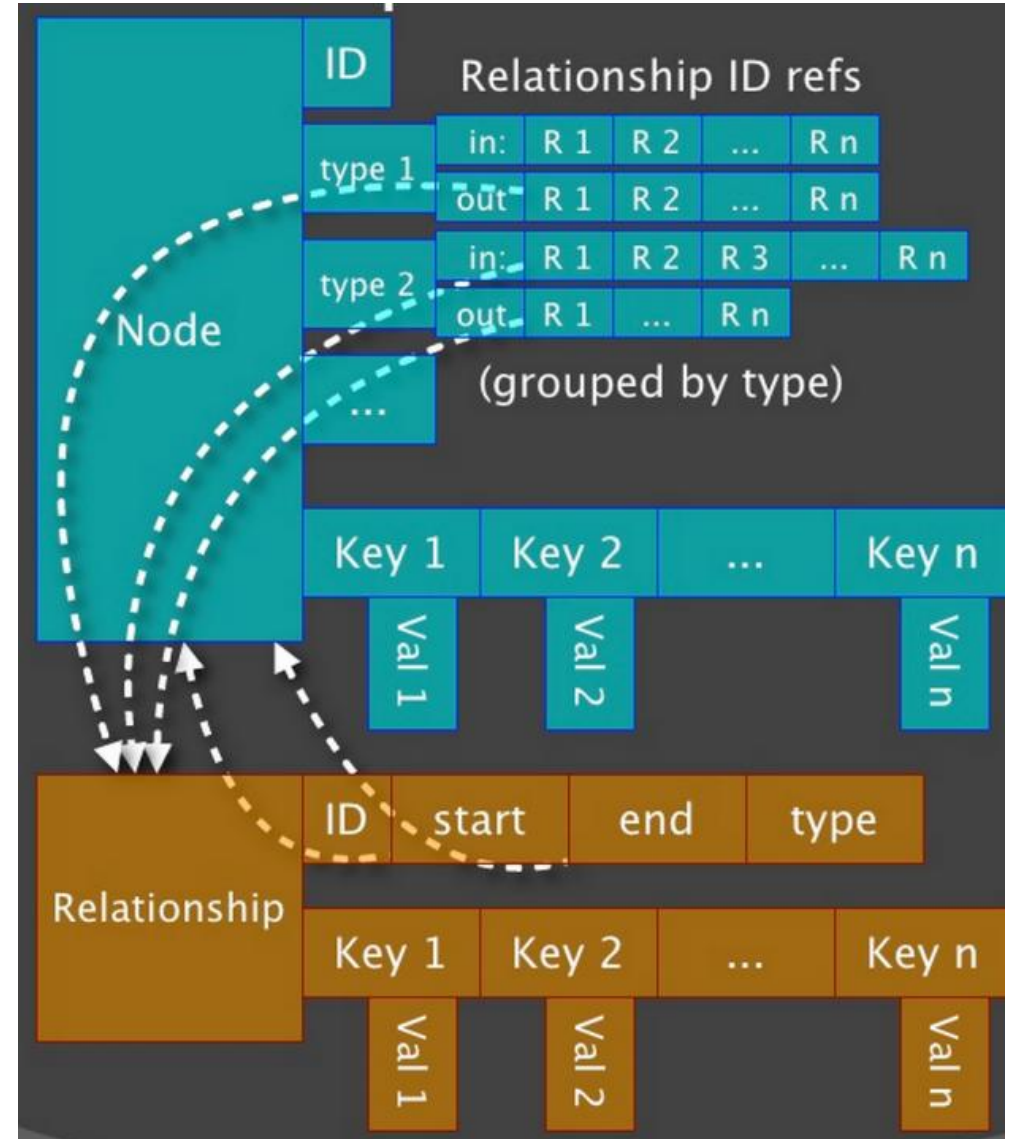
Quelques faits sur Neo4J

Robinson et al., 2013



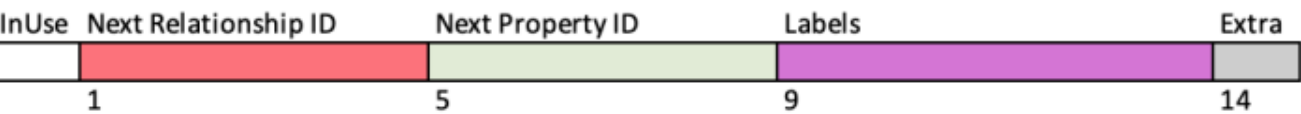
Le cache

- Pointeurs au lieu de listes de contrôle
- Enregistrements de taille fixe
- Les jointure sont faite à la création
- Parcourir cette structure de données autant de fois qu'il faut

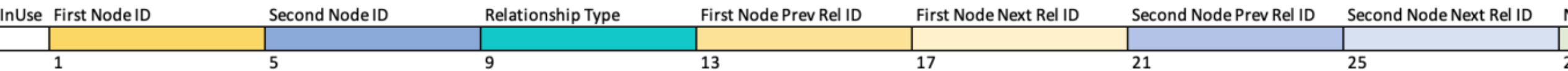


Layout du registre de stockage

Node(15 Bytes)



Relationship (34 Bytes)



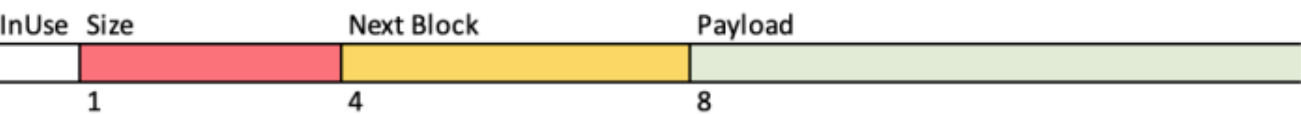
RelationshipGroup (25 Bytes)



Property Record (41 Bytes)

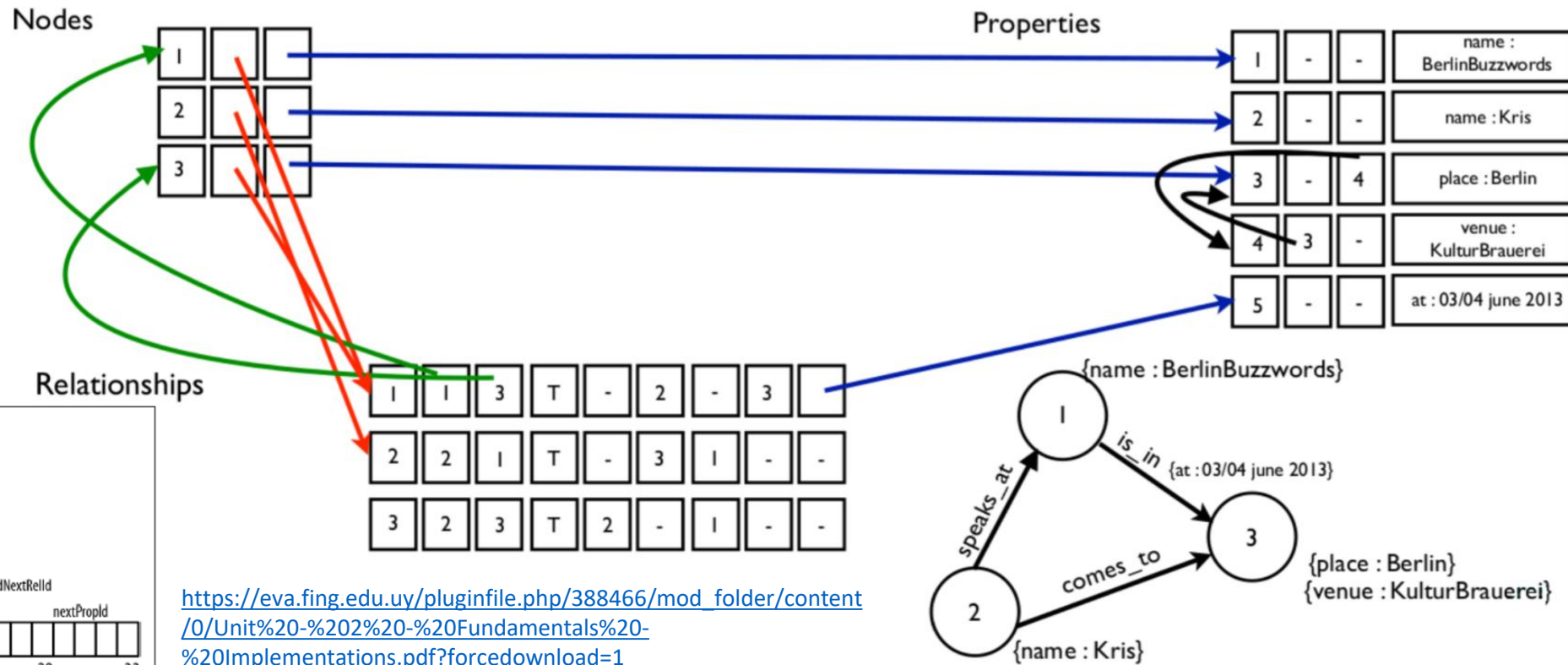


Dynamic Record



Stockage des graphes en fichiers - Neo4J

- Plusieurs fichiers sont physiquement utilisés pour le stockage
- Données stockées comme des listes d'enregistrements liées
- Nœuds, propriétés et relations sont stockés séparément



https://eva.fing.edu.uy/pluginfile.php/388466/mod_folder/content/0/Unit%20-%20Fundamentals%20-%20Implementations.pdf?forcedownload=1

Cypher - Clauses générales - Neo4J

- Un langage déclaratif permettant de requêter et mettre à jour le graphe.
- Inspiré du SQL, on y retrouve beaucoup de concepts familiers comme les clauses WHERE, ORDER BY, LIMIT...
- L'objectif est de permettre à l'utilisateur de définir des motifs « pattern », qui seront par la suite recherchés dans tout le graphe

Exemple :

Ann aime Dan

```
CREATE (:Personne{name:"Ann"}) -[:AIME]-> (:Personne{name:"Dan"})
```

Cypher - Clauses générales - Neo4J

- (:NOEUD) : est utilisé pour définir le noeud et ses propriétés.
- - [:RELATION] - : est utilisé pour définir la relation entre les nœuds et ses éventuelles propriétés.
- < ou > : avant ou après la relation est utilisé pour définir l'orientation de la relation.
- Pour interroger Neo4j avec Cypher on utilise MATCH qui est l'équivalent du SELECT en SQL.
 - MATCH : recherche un élément et retourne une table ou un sousgraphe
 - DISTINCT : élimine la redondance
 - OPTIONAL MATCH : relation optionnelle (jointure externe en SQL)
 - WHERE : sélection Supporte les expressions régulières sur les chaînes
 - RETURN : retourne le sous-graphe ou la table

Cypher - Clauses générales - Neo4J

- (:NOEUD) : est utilisé pour définir le noeud et ses propriétés.
- - [:RELATION] - : est utilisé pour définir la relation entre les nœuds et ses éventuelles propriétés.
- < ou > : avant ou après la relation est utilisé pour définir l'orientation de la relation.
- Pour interroger Neo4j avec Cypher on utilise MATCH qui est l'équivalent du SELECT en SQL.
 - MATCH : recherche un élément et retourne une table ou un sousgraphe
 - DISTINCT : élimine la redondance
 - OPTIONAL MATCH : relation optionnelle (jointure externe en SQL)
 - WHERE : sélection Supporte les expressions régulières sur les chaînes
 - RETURN : retourne le sous-graphe ou la table

Cypher - Clauses générales - Neo4J

➤ **Nœud**

- Nœud anonyme: `()`
- Nœud nommé: `(x)`
- Nœud avec un label spécifique: `(:label)`

➤ **Relations**

- Relation anonyme: `-[]->`
- Relation nommée: `-[r]->`
- Relation nommée avec un label spécifique: `-[r:t]->`
- 2 nœuds avec une relation: `(a)-[r]->(b)`

- Les propriétés peuvent être spécifiées via `{}`, e.g., `(x {city: "Chicago"})`
- Un motif peut combiner plusieurs nœuds et relations

Cypher - Clauses générales - Neo4J (Exemples)

```
1 // Renvoie tous les noeuds d'une base
2 MATCH (n) RETURN n
3 // Renvoie tous les noeuds de label Product
4 MATCH (n:Product) RETURN n
5 // Renvoie tous les noeuds de label Product
6 // qui ont pour propriété name : Neo4j in a nutshell
7 MATCH (n:Product{name : "Neo4j in a nutshell"})
8 RETURN n
9 // Idem
10 MATCH (n:Product)
11 WHERE n.name = "Neo4j in a nutshell"
12 RETURN n
13 // Retourne un produit et certaines de ses propriétés
14 MATCH (prod:Product { name: "NoSQL Distilled" })
15 RETURN prod.name;
```

Cypher - Neo4J

```
1 // Retourne les commandes de Martin
2 MATCH (a:Customer { name: 'Martin' })-[: ]→(x)
3 RETURN x
4 // Retourne les clients triés selon leur nom
5 MATCH (customer:Customer)
6 RETURN customer ORDER BY customer.name;
7 // Tous les clients dont le nom termine par in
8 MATCH (customer:Customer)
9 WHERE customer.name =~ ".*in$"
10 RETURN customer.name;
```

Cypher - Clauses d'Écriture

- CREATE : crée un nœud ou une relation
- MERGE : crée un nœud ou une relation si elle n'existe pas déjà
- SET : modifie/ajoute des données/labels
- REMOVE : supprime des labels et des propriétés
- DELETE : supprime des éléments du graphe

Cypher - Exemples création

```
1 // Peuple une base
2 CREATE (prod1:Product {name : "NoSQL Distilled"})
3 CREATE (ord1:Orders)
4 CREATE (ord1)-[:ORDERITEM { Price : 32.45 }]->(prod1)
5 // OU
6 CREATE (ord1:Orders) -[:ORDERITEM { Price : 32.45 }] →
  (prod1:Product {name : "NoSQL Distilled"})
7
8
9 // Charge une table depuis un CSV volumineux
10 USING PERIODIC COMMIT
11 LOAD CSV WITH HEADERS FROM
  'http://neo4j.com/docs/2.2.5/csv/artists-with-headers.csv' AS line
12 CREATE (:Artist { name: line.Name, year: toInt(line.Year)})
```

Cypher - Exemples mise à jour / suppression

```
1 // Fixe la propriété name des noeuds dont la propriété name valait
  "NoSQL Distilled" à "NoSQL Distilled v1"
2 MATCH (n) WHERE n.name = "NoSQL Distilled" SET n.name = "NoSQL
  Distilled v1";
3
4 // Suppression d'une propriété
5 MATCH (n) -[r:ORDERITEM]→ (m)
6 REMOVE r.Price
7
8 // Suppression d'une relation
9 MATCH (m) -[r:ORDERITEM]→ (n:Product {name : "NoSQL Distilled"})
10 DELETE r
11
12 // Suppression d'un noeud et de ses relations
13 MATCH (n:Product {name : "NoSQL Distilled"}) DETACH
14 DELETE n
15
16 // Nettoyage complet de la base
17 MATCH (n) DETACH DELETE n;
```

Cypher – Schéma

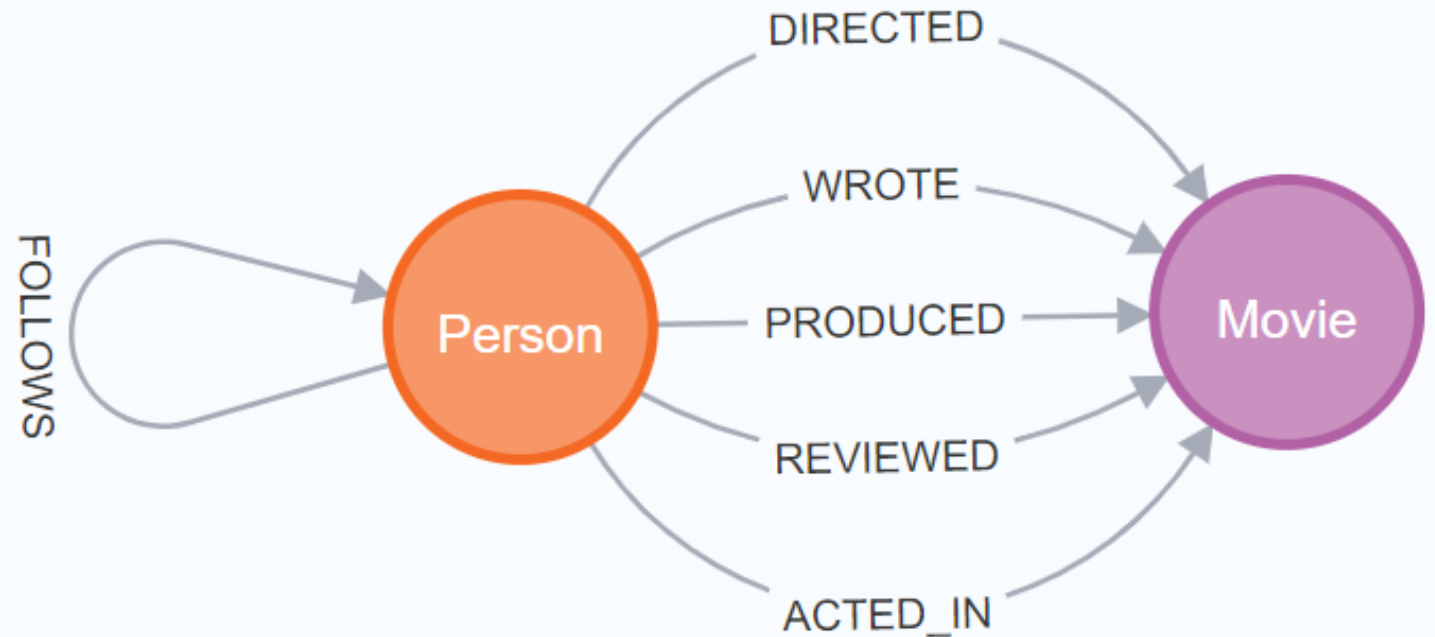
```
$ call db.schema.visualization()
```

Graph

Table

Text

Code



Merci !